

# Guia 1

# Sistema de Gestão Hospitalar

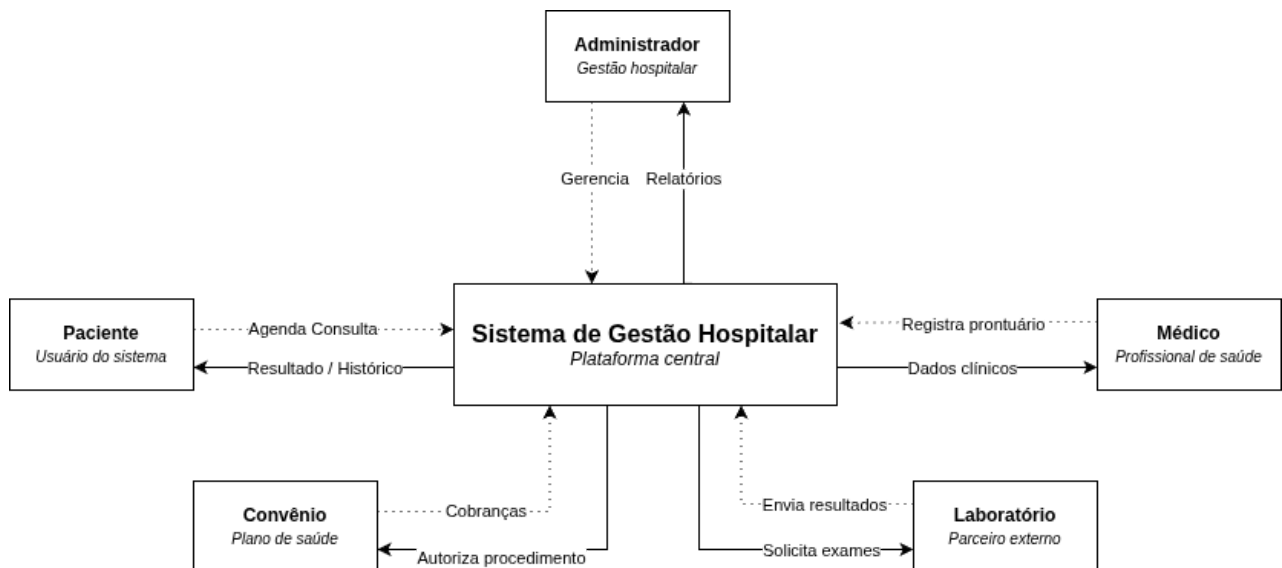
## Modelo C4 — Diagrama de Contexto

Integrantes: João Victor Fróis, João Victor Soares, Guilherme Henrique, Lucas Gabriel Ferreira, Pedro Canuto

## Introdução

O Sistema de Gestão Hospitalar trata-se de uma plataforma focada em centralizar os processos de um hospital. O sistema é responsável por coordenar as interações entre diversos agentes: pacientes, médicos, administradores, convênios e laboratórios. Cada um deles possuirá atribuições específicas que, em conjunto, sintetizarão as operações da instituição de maneira fluida, moderna e funcional. Este documento apresenta a estrutura da ferramenta através do modelo C4, que divide o sistema em quatro níveis de abstração hierárquicos: Contexto, Contêiner, Componente e Código.

## Nível 1 – Diagrama de Contexto



## Atores Externos Identificados

- Médico – registra prontuários, prescrições e solicita exames ao laboratório
- Administrador – gerencia leitos, equipes, recursos hospitalares e gera relatórios
- Convênio – autoriza procedimentos e recebe informações de cobrança
- Laboratório Externo – recebe solicitações de exames e devolve resultados

## Descrição do Diagrama

O Diagrama de Contexto apresenta o Sistema de Gestão Hospitalar como uma caixa central, acompanhada de cinco atores externos. O sistema funciona como um mediador central que gerencia os processos e a rede de interações entre os pacientes, médicos, administradores, laboratórios e convênios. Cada um dos agentes interage com o sistema de maneira distinta: pacientes o usam para gerir sua jornada de atendimento; médicos usam como ferramenta de registro clínico; administradores o usam para monitoramento operacional; convênios se integram para autorização e cobrança; e laboratórios se conectam para troca de resultados.

## Decisões Arquiteturais – Nível 1

O sistema foi desenhado como uma caixa única e centralizada porque ele é o coordenador de todos os processos hospitalares. Cada ator é externo pelos seguintes motivos:

- Paciente é externo: porque é um usuário do sistema, não parte da infraestrutura do hospital.
- Médico é externo: porque é um profissional que interage com o sistema, mas sua especialidade pertence ao domínio clínico, não à aplicação.
- Administrador é externo: porque é um operador que gerencia recursos, mas não faz parte do sistema em si. Ele apenas comanda o sistema, não o fluxo do hospital.
- Convênio é externo: porque os planos de saúde têm infraestrutura e regras independentes. O hospital se integra a elas via API, não as incorpora internamente.
- Laboratório é externo: porque testes especializados exigem equipamentos sofisticados e perícia que um hospital generalista não justifica manter internamente.

## Referências e Fonte

Modelo C4 – Simon Brown. Disponível em: <https://c4model.com>

# Guia 2

## Nível 2 - Diagrama de Containers



## Contêineres identificados

### Front-ends (interfaces visuais)

- **Web App** - portal acessado pelo navegador; permite ao paciente agendar consultas, fazer check-in e visualizar histórico médico.
- **Mobile App (iOS/Android)** - alternativa móvel com as mesmas funções do Web App, voltada ao uso cotidiano do paciente.
- **Painel Médico** - interface exclusiva para médicos; exibe prontuários, prescrições e resultados de exames.
- **Painel Administrativo** - interface para o administrador gerenciar leitos, equipes e gerar relatórios operacionais.

## Backend

- **API Backend** - núcleo do sistema; concentra todas as regras de negócio (agendamento, prontuário, autorização de procedimentos), autenticação/autorização de usuários e orquestra a comunicação entre os demais contêineres.

## Dados e serviços internos

- **Banco de Dados Relacional** - armazena pacientes, médicos, agendamentos, prontuários e registros administrativos.
- **Serviço de Notificações** - responsável por enviar confirmações de consulta, lembretes e alertas via e-mail e SMS.

## Sistemas externos

- **API de Convênio** - integração externa para autorização de procedimentos e faturamento.
- **API de Laboratório** - integração para envio de solicitações de exame e recebimento de resultados.

## Fluxos principais

| De                           | Para                    | Motivo                                                               |
|------------------------------|-------------------------|----------------------------------------------------------------------|
| Web App / Mobile App         | API Backend             | Toda ação do paciente passa pela API                                 |
| Painel Médico / Painel Admin | API Backend             | Mesma API centraliza regras de todos os perfis                       |
| API Backend                  | Banco de Dados          | Leitura e escrita de todos os dados clínicos e operacionais          |
| API Backend                  | Serviço de Notificações | Dispara alertas após eventos (consulta confirmada, exame disponível) |
| API Backend                  | API de Convênio         | Solicita autorização de procedimentos e envia dados de cobrança      |
| API Backend                  | API de Laboratório      | Envia pedidos de exame, recebe resultados de volta                   |

## Decisões arquiteturais – Nível 2

**Por que separar os front-ends?** Pacientes, médicos e administradores têm necessidades muito diferentes. Manter painéis separados permite controle de acesso granular, interfaces adaptadas a cada perfil e implantação independente.

**Por que uma única API Backend?** Centralizar as regras de negócio evita duplicação e garante consistência — um agendamento feito pelo app mobile segue as mesmas validações que um feito pelo portal web. A API também é o único ponto de contato com sistemas externos, facilitando auditoria e segurança.

**Por que um serviço de notificações separado?** Notificações são um canal de saída assíncrono. Isolá-las impede que falhas no envio de e-mail/SMS impactem o fluxo principal de atendimento.

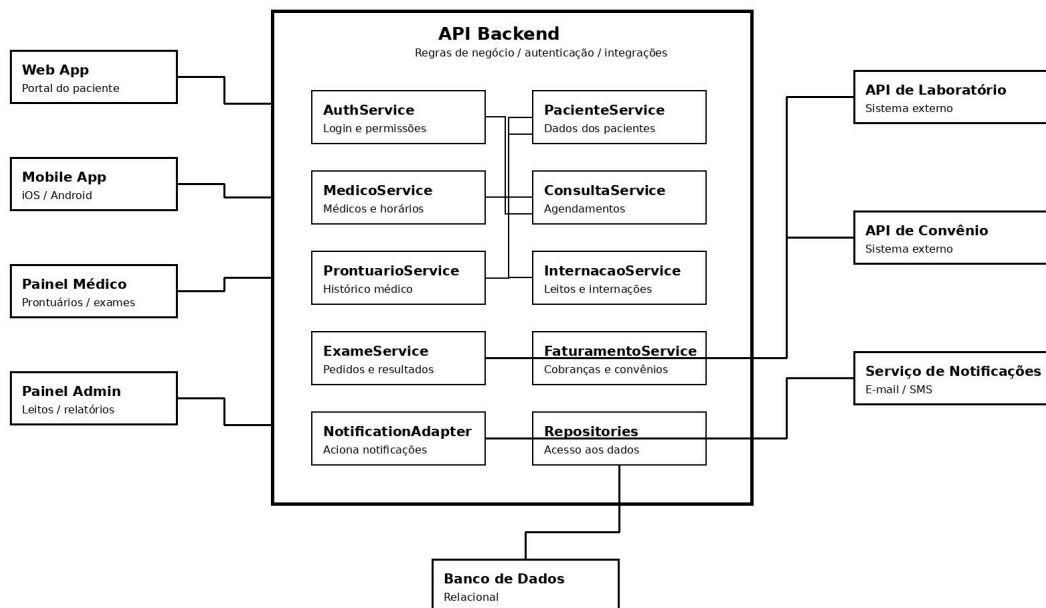
**Por que os convênios e laboratórios são externos?** Como definido no Nível 1, ambos possuem infraestrutura e regras próprias. O hospital se integra via API, sem incorporar a lógica dessas entidades internamente — o que reduz a complexidade do sistema e mantém o acoplamento fraco.

# Guia 3



## Nível 3 - Diagrama de Componentes

C4 - Nível 3: Diagrama de Componentes da API Backend



›

### Componentes identificados:

- AuthService – autenticação e controle de acesso dos usuários.
- PacienteService – gerenciamento de informações dos pacientes.
- MedicoService – gerenciamento de médicos e especialidades.
- ConsultaService – controle de consultas e agendamentos.
- ProntuarioService – gerenciamento de prontuários médicos.
- InternacaoService – controle de internações e leitos.
- ExameService – integração com laboratórios externos.
- FaturamentoService – integração com convênios e cobranças.
- NotificationAdapter – acionamento do serviço de notificações.
- Repositories – acesso e persistência dos dados no banco.

### Fluxos principais:

- ConsultaService -> PacienteService: consulta informações do paciente.

- ConsultaService -> MedicoService: verifica disponibilidade do médico.
- ExameService -> API de Laboratório: envio e recebimento de exames.
- FaturamentoService -> API de Convênio: autorização e cobrança de procedimentos.
- NotificationAdapter -> Serviço de Notificações: envio de alertas e lembretes.
- Repositories -> Banco de Dados: armazenamento e recuperação das informações.

## Decisões arquiteturais

A API Backend foi dividida em componentes especializados para separar responsabilidades e facilitar a manutenção do sistema. Cada componente é responsável por uma área específica do domínio hospitalar, como consultas, prontuários, internações e faturamento.

As integrações com laboratórios e convênios foram isoladas em componentes próprios, reduzindo o acoplamento com sistemas externos. O acesso ao banco de dados foi centralizado na camada de Repositories, promovendo maior organização e reutilização de código.

Essa divisão torna o sistema mais modular, escalável e fácil de evoluir ao longo do tempo.

# Guia 4

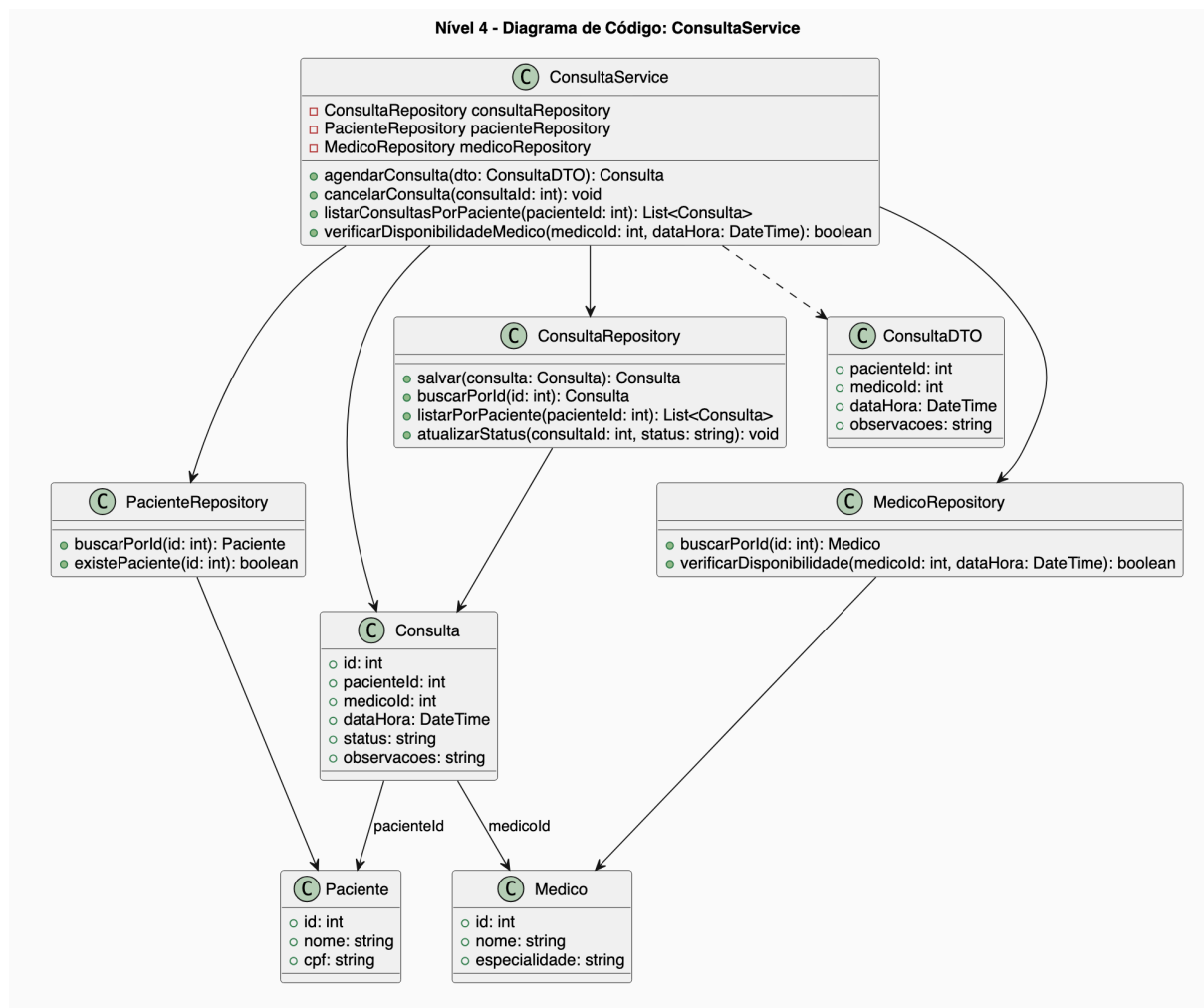
## Nível 4 - Diagrama de Código

› Componente escolhido: **ConsultaService**

Classes identificadas:

- **ConsultaService** – responsável pelas regras de negócio de agendamento, cancelamento e consulta de horários.
- **ConsultaRepository** – responsável por acessar e persistir dados de consultas no banco.
- **PacienteRepository** – consulta dados do paciente antes do agendamento.
- **MedicoRepository** – verifica dados e disponibilidade do médico.
- **ConsultaDTO** – objeto usado para transferir dados entre a API e o serviço.
- **Consulta** – entidade principal que representa uma consulta médica.

UML da classe:



## Atributos principais:

### Consulta

- id
- pacienteld
- medicold
- dataHora
- status
- observacoes

### ConsultaDTO

- pacienteld
- medicold
- dataHora
- observacoes

## Métodos principais:

### ConsultaService

- agendarConsulta(ConsultaDTO dto)
- cancelarConsulta(int consultald)
- listarConsultasPorPaciente(int pacienteld)
- verificarDisponibilidadeMedico(int medicold, DateTime dataHora)

### ConsultaRepository

- salvar(Consulta consulta)
- buscarPorId(int id)
- listarPorPaciente(int pacienteld)
- atualizarStatus(int consultald, string status)

## Padrões utilizados:

- **Repository** – separa as regras de negócio do acesso ao banco de dados.
- **DTO** – evita expor diretamente as entidades internas da aplicação.
- **Injeção de Dependência** – permite que o **ConsultaService** utilize os repositórios sem criar dependência rígida entre as classes.

## Fluxos principais:

- **ConsultaService -> PacienteRepository**: valida se o paciente existe.
- **ConsultaService -> MedicoRepository**: verifica se o médico existe e está disponível.
- **ConsultaService -> ConsultaRepository**: salva, busca ou atualiza consultas.
- **ConsultaRepository -> Banco de Dados**: persiste e recupera os dados da consulta.

## Decisões arquiteturais

O componente **ConsultaService** foi escolhido por ser uma parte central do sistema hospitalar, pois conecta pacientes, médicos e horários de atendimento. A separação entre **ConsultaService**, **ConsultaRepository**, **ConsultaDTO** e **Consulta** facilita a manutenção e evita que regras de negócio fiquem misturadas com acesso ao banco.

O uso do padrão **Repository** melhora a organização do código e permite alterar a forma de persistência sem afetar diretamente a lógica de agendamento. Já o uso de **DTOs** torna a comunicação da API mais segura e controlada, expondo apenas os dados necessários.

Essa estrutura torna o componente mais modular, testável e coerente com o diagrama de componentes do Nível 3.

# Guia 5

## Justificativa Arquitetural

A arquitetura do Sistema de Gestão Hospitalar foi estruturada utilizando o Modelo C4 para representar o sistema em diferentes níveis de abstração, permitindo uma visão clara tanto do contexto geral quanto dos componentes internos responsáveis pela execução das funcionalidades.

A divisão entre aplicações Web, Mobile, Painei Médico e Painei Administrativo foi adotada para atender às necessidades específicas de cada perfil de usuário. Essa separação permite interfaces mais adequadas para pacientes, médicos e administradores, além de facilitar a manutenção e evolução independente de cada módulo.

A utilização de uma API Backend centralizada garante a padronização das regras de negócio e a consistência dos dados em todo o sistema. Dessa forma, operações como agendamento de consultas, acesso a prontuários e gerenciamento administrativo seguem os mesmos processos de validação, independentemente da interface utilizada.

A integração com convênios e laboratórios foi realizada por meio de APIs externas, já que esses sistemas normalmente possuem seus próprios ambientes e bancos de dados. Dessa forma, o sistema hospitalar consegue trocar informações importantes sem precisar armazenar ou controlar todos esses processos internamente.

No nível de código, foram utilizados padrões como Repository, DTO e Injeção de Dependência. Esses padrões promovem melhor organização do software, separação de responsabilidades, facilidade de testes e maior flexibilidade para futuras manutenções e expansões do sistema.

A arquitetura foi planejada considerando requisitos de segurança e privacidade, fundamentais em sistemas de saúde. A centralização do acesso aos dados pela API permite maior controle sobre autenticação, autorização e auditoria das operações realizadas, contribuindo para a conformidade com os princípios da LGPD e para a proteção das informações sensíveis dos pacientes.

O Modelo C4 foi importante para representar a arquitetura do sistema de forma organizada, permitindo visualizar desde os usuários e sistemas externos até os componentes internos e classes responsáveis pelas funcionalidades. Isso facilitou a compreensão das responsabilidades de cada parte do sistema e da forma como elas se relacionam.